

Criptografía — Actividad: Security Audit Report

Barrios Aguilar Dulce Michelle
Caballero Martínez Sergio Jair
Contreras Colmenero Emilio Sebastian
Martínez López Evan Emiliano

Facultad de Ingeniería · UNAM 2026-2

1. Introducción

Para esta actividad lo que hicimos fue una auditoría de seguridad a nuestro propio sistema, el SDDV (Secure Digital Document Vault), que está en <https://github.com/millyx/Cryptography>. La idea era checar si nuestro sistema realmente detecta cuando alguien modifica algo en los archivos que protege, o si se le pasan los cambios sin avisar.

La actividad pedía probar cinco cosas distintas: modificar la metadata, la lista de destinatarios, el nonce, la firma y los identificadores de llave. Para cada una hay que decir qué cambiamos, qué pasó cuando intentamos abrir el archivo modificado, si el sistema se dio cuenta y qué propiedad de seguridad se ve afectada en cada caso.

Las pruebas las hicimos sobre los tres módulos principales del sistema: `crypto/aead.py` (que maneja el cifrado de los archivos), `crypto/hybrid.py` (que se encarga del cifrado para varios destinatarios al mismo tiempo) y `crypto/signatures.py` (que es el que pone y verifica las firmas).

2. Metodología

Lo que hicimos para cada prueba fue básicamente lo mismo: primero generamos un archivo cifrado normal, sin tocar nada, para tener una referencia. Después modificamos un byte específico del archivo, dependiendo de qué parte queríamos atacar. Luego intentamos descifrar o verificar ese archivo modificado y vimos qué pasaba (si el sistema lo aceptaba o lo rechazaba). Al final apuntamos qué excepción nos lanzó Python.

Para que todo quedara reproducible y no tener que andar haciendo las pruebas a mano cada vez, escribimos un script llamado `audit_tampering.py` que está en la raíz del repositorio. Ese script hace los cinco ataques uno tras otro, y para cada uno enseña el archivo en hexadecimal antes y después de la modificación, además del traceback completo cuando algo falla. La salida completa de ese script la pegamos al final del reporte como anexo, y también incluimos una captura de cómo se ve cuando lo corres en la terminal (en la sección 4).

3. Pruebas de auditoría

3.1 Modificación de Metadata

Descripción. En esta prueba lo que quisimos checar es qué pasa si alguien modifica el nombre del archivo dentro del contenedor cifrado. El nombre del archivo va guardado dentro de lo que se llama AAD (datos asociados que se autentican pero no se cifran), y la idea es que si se cambia, el sistema lo debería detectar al momento de descifrar.

Qué se modificó. Cambiamos el byte número 16 del archivo binario, que es justo donde empieza el nombre. La primera letra de `contrato.pdf` (la `c`, que en hex es `0x63`) la cambiamos a `b` (`0x62`) haciendo un XOR con `0x01`. O sea, solo cambiamos un bit, lo más mínimo posible.

Comportamiento observado. El sistema lanzó la excepción `cryptography.exceptions.InvalidTag` y no nos dejó ver ni un solo byte del contenido original. La salida completa del test se ve así:

```
=====
TEST 1: Modificacion de METADATA (filename en AAD del SDDV)
=====

--- FASE 1 - GENERACION DE CONTENEDOR LIMPIO ---
plaintext..... b'Documento confidencial de auditoria'
filename..... 'contrato.pdf'
algoritmo..... AES-256-GCM
llave generada..... cf0bca6e21afc44727b0125e819d664c543d73671204d8efbc9f8c651d43f8bc
contenedor (bytes)..... 95

hex dump (contenedor SDDV original, 95 bytes total):
00000000 53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 0c |SDDV.....i.S...|
00000010 63 6f 6e 74 72 61 74 6f 2e 70 64 66 67 e5 14 62 |contrato.pdfg..b|
00000020 ca 83 6f 0d 41 33 32 30 00 00 00 23 3c f0 ad 74 |..o.A320...#<..t|
00000030 1f eb 7f d8 bf 26 25 43 96 dc e0 bb ae 85 c2 e8 |.....&%C.....|
00000040 2b 69 48 bc e0 6e c2 95 0c 4b 71 a2 bb 71 ac 22 |+iH..n...Kq..q."|
00000050 c5 6b 57 8d 08 4b ea 56 37 d5 07 02 5c 00 43   |.kW..K.V7...\C|

--- FASE 2 - VERIFICACION DEL HAPPY PATH ---
primero confirmamos que el contenedor LIMPIO se descifra bien:
plaintext recuperado..... b'Documento confidencial de auditoria'
[OK] happy path funciona correctamente

--- FASE 3 - APLICAR MODIFICACION MALICIOSA ---

>> MODIFICACION en byte[16]:
    antes:  0x63  (01100011)  'c'
    despues: 0x62  (01100010)  'b'
    XOR:     0x01  (00000001)
    descripcion: flip de bit en filename (offset 16)

hex dump (contenedor SDDV ADULTERADO, 95 bytes total):
00000000 53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 0c |SDDV.....i.S...|
00000010 62 6f 6e 74 72 61 74 6f 2e 70 64 66 67 e5 14 62 |bontrato.pdfg..b|
00000020 ca 83 6f 0d 41 33 32 30 00 00 00 23 3c f0 ad 74 |..o.A320...#<..t|
```

...

--- INTENTO DE DESCIFRADO / VERIFICACION ---

llamada: decrypt_file(tampered_container, key)

[EXCEPCION CAPTURADA]

tipo..... cryptography.exceptions.InvalidTag

traceback completo:

File "crypto/aead.py", line 192, in decrypt_file

plaintext = cipher.decrypt(nonce, ciphertext + tag, header)

cryptography.exceptions.InvalidTag

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

Antes de hacer el ataque verificamos que el archivo original se pudiera descifrar bien (lo que llamamos “happy path”) y sí funcionó: recuperó el texto exacto que metimos. Pero después, con el byte cambiado, ya no hubo manera de abrirlo.

¿El sistema detectó la modificación? Sí, sin problema. Apenas le pasamos el archivo modificado a la función `decrypt_file`, lanzó la excepción y se detuvo ahí mismo. No nos dio el contenido del archivo ni siquiera parcialmente.

Propiedad de seguridad afectada. *Integridad.* Lo que protege esto es que nadie pueda meterle mano al archivo cifrado y que el dueño no se entere. El sistema usa una etiqueta llamada TAG que se genera junto con el cifrado y que cubre tanto el contenido como los metadatos (el nombre, la fecha, etc.). Si cualquier byte cambia, el TAG ya no coincide y el sistema lo rechaza.

3.2 Modificación de Lista de Destinatarios

Descripción. Cuando ciframos un archivo para varias personas (por ejemplo, para Alice y Bob al mismo tiempo), el sistema guarda una lista con los identificadores de cada destinatario autorizado. Lo que queríamos ver aquí es qué pasa si alguien modifica esa lista, en concreto el identificador de Alice. Si lo cambiamos, ¿Alice todavía puede abrir el archivo o no?

Qué se modificó. Le cambiamos un bit al primer byte del fingerprint (identificador) de Alice dentro de la lista. Otra vez fue un XOR con 0x01, o sea el cambio más chiquito posible. Pasó de 0x94 a 0x95.

Comportamiento observado. El sistema rechazó a Alice y nos dijo que no estaba autorizada, aunque sí lo estuviera. Eso es porque al modificar el fingerprint, ya no coincide con el de su llave. Esta es la salida del test:

```
=====
TEST 2: Modificacion de LISTA DE DESTINATARIOS (fingerprint en AAD)
=====
```

```
--- FASE 1 - GENERACION DE LLAVES Y CONTENEDOR HIBRIDO ---
```

```
Alice X25519 FP..... 94b9eea73e4ea73f27d61e5e9b84662ff7273e0bae613aa2f037a7869712c647
Bob   X25519 FP..... f96fa7f343de01c73c5feda83819edd576c6e3baaaf77057f2b7644e7c5585f1
plaintext..... b'Documento compartido entre Alice y Bob'
contenedor (bytes)..... 343
destinatarios..... 2
```

```
--- FASE 2 - VERIFICACION DEL HAPPY PATH (Alice y Bob descifran) ---
```

```
Alice recupera..... b'Documento compartido entre Alice y Bob'
Bob recupera..... b'Documento compartido entre Alice y Bob'
[OK] ambos destinatarios descifran correctamente
```

```
--- FASE 3 - APLICAR MODIFICACION MALICIOSA ---
```

```
>> MODIFICACION en byte[25]:
```

```
antes: 0x94 (10010100)
```

```
despues: 0x95 (10010101)
```

```
XOR: 0x01 (00000001)
```

```
descripcion: flip de bit en primer byte del fingerprint de Alice
```

```
hex dump (contenedor SDDH ADULTERADO, 343 bytes total):
```

```
00000000 53 44 44 48 01 01 00 00 00 00 69 fa 53 08 00 07 |SDDH.....i.S...|
00000010 64 6f 63 2e 70 64 66 00 02 95 b9 ee a7 3e 4e a7 |doc.pdf.....>N.|
...
```

```
--- INTENTO DE DESCIFRADO / VERIFICACION ---
```

```
llamada: decrypt_for_recipient(tampered, alice_priv)
```

```
[EXCEPCION CAPTURADA]
```

```
tipo..... builtins.ValueError
```

```
mensaje..... 'Este destinatario no esta autorizado en el contenedor'
```

```
traceback completo:
```

```
File "crypto/hybrid.py", line 359, in decrypt_for_recipient
```

```
raise ValueError("Este destinatario no esta autorizado en el contenedor")
ValueError: Este destinatario no esta autorizado en el contenedor
```

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

Antes del ataque comprobamos que tanto Alice como Bob podían descifrar normalmente, lo cual sí funcionó. Después de modificar el fingerprint, cuando Alice intenta entrar el sistema no encuentra su identificador en la lista y le dice que no está autorizada.

¿El sistema detectó la modificación? Sí. La función que descifra busca el identificador de Alice en la lista, y como ya no coincide (por el bit que cambiamos), simplemente le manda un `ValueError` y no intenta nada más.

Propiedad de seguridad afectada. *Integridad y control de acceso.* Por un lado, modificar la lista hace que el sistema rechace a quien sí debería tener acceso (eso es control de acceso). Y por otro lado, aunque el atacante quisiera seguir adelante con la lista modificada, igual fallaría porque la lista forma parte del AAD y cualquier cambio invalida el TAG general del archivo.

3.3 Modificación de Nonce

Descripción. El nonce es un valor aleatorio de 12 bytes que se genera nuevo cada vez que se cifra algo. Sirve para que aunque cifres dos veces el mismo archivo con la misma llave, el resultado se vea totalmente distinto. Si alguien lo modifica en el archivo cifrado, lo más seguro es que ya no se pueda recuperar nada. Eso es lo que queríamos comprobar.

Qué se modificó. Le hicimos un XOR con 0xFF al primer byte del nonce. Esto cambia los 8 bits de ese byte, es decir, lo invierte completo. El byte pasó de 0x71 a 0x8e.

Comportamiento observado. Igual que el primer test, el sistema lanzó InvalidTag y no nos devolvió nada. Esta es la salida completa:

```
=====
TEST 3: Modificacion de NONCE (DEM nonce de 96 bits)
=====

--- FASE 1 - GENERACION DE CONTENEDOR LIMPIO ---
plaintext..... b'Mensaje protegido por AES-256-GCM'
llave..... d6a8bc0306e7d6434d1d33dc0e7e17c4e65f0fc679e6ea9c8851e5d64a26cb4e
contenedor bytes..... 88
nonce offset..... 23
nonce original..... 71e9580c4bfe5584886bda28

hex dump (contenedor SDDV original, 88 bytes total):
00000000  53 44 44 56 01 01 00 00  00 00 69 fa 53 08 00 07  |SDDV.....i.S...|
00000010  64 6f 63 2e 74 78 74 71  e9 58 0c 4b fe 55 84 88  |doc.txtq.X.K.U..|
00000020  6b da 28 00 00 00 21 94  75 aa ef 01 df 87 32 34  |k.(...!.u.....24|
...
(los bytes resaltados son los 12 bytes del nonce)

--- FASE 2 - APLICAR MODIFICACION AL NONCE ---

>> MODIFICACION en byte[23]:
    antes:  0x71  (01110001)  'q'
    despues: 0x8e  (10001110)
    XOR:     0xff (11111111)
    descripcion: flip total (XOR 0xFF) del primer byte del nonce
nonce modificado..... 8ee9580c4bfe5584886bda28

--- INTENTO DE DESCIFRADO / VERIFICACION ---
llamada: decrypt_file(tampered_container, key)

[EXCEPCION CAPTURADA]
tipo..... cryptography.exceptions.InvalidTag
traceback completo:
  File "crypto/aead.py", line 192, in decrypt_file
    plaintext = cipher.decrypt(nonce, ciphertext + tag, header)
  cryptography.exceptions.InvalidTag

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext
```

¿El sistema detectó la modificación? Sí. El TAG ya no coincide porque el sistema usa el nonce para hacer las cuentas internamente, y al cambiarlo todo se desordena. Por eso lanza `InvalidTag`.

Propiedad de seguridad afectada. *Confidencialidad e integridad.* Si el sistema dejara descifrar con un nonce modificado, lo que saldría sería puro ruido, basura, no el archivo original. Pero además, el sistema se da cuenta y de plano no devuelve nada. Esto también nos sirvió para confirmar que el sistema genera un nonce nuevo cada vez (con `os.urandom`), porque si reusara el nonce sería un problema grande de seguridad en GCM.

3.4 Modificación de Firma Digital (Ed25519)

Descripción. Cuando alguien firma un archivo con Ed25519, al final del archivo se agregan 64 bytes que son la firma. Lo que queríamos probar es qué pasa si alguien le mueve aunque sea un bit a esa firma. La idea es que con cualquier cambio, por mínimo que sea, la firma ya no debería verificarse.

Qué se modificó. Cambiamos un byte dentro de los 64 bytes de la firma. Otra vez con un XOR con 0x01, así que solo un bit. El byte pasó de 0xe7 a 0xe6.

Comportamiento observado. El sistema rechazó la firma con `InvalidSignature` y, como esta es la primera revisión que hace el sistema antes de descifrar nada, ni siquiera llegó a intentar abrir el archivo. La salida es esta:

```
=====
TEST 4: Modificacion de FIRMA DIGITAL (Ed25519)
=====
```

```
--- FASE 1 - GENERACION DE LLAVES Y CONTENEDOR FIRMADO ---
```

```
Alice Ed25519 pub..... 64c6a8867d3206d6d99870d3eb3be5436d04a1246e6850f50dfd20563beac3f0
Alice fingerprint..... 32177b9c961e0abf4f01ac55b49f4fea2d7036311943160bda73995c927fb731
contenedor SDDV..... 82 bytes
contenedor firmado..... 182 bytes (= SDDV + 100 bytes footer)
layout footer..... SIGS(4) + FINGERPRINT(32) + SIGNATURE(64)
firma Ed25519 (hex)..... bebe1e9aa788a3f5e037273d34e045311ddcc6598a818791efb0aa046e1176b7
```

```
--- FASE 2 - VERIFICACION DEL HAPPY PATH ---
```

```
verify_container retorno 82 bytes (= SDDV original)
plaintext recuperado..... b'Documento firmado por Alice'
[OK] firma valida + descifrado correcto
```

```
--- FASE 3 - APLICAR MODIFICACION A LA FIRMA ---
```

```
>> MODIFICACION en byte[172]:
  antes:  0xe7  (11100111)
  despues: 0xe6  (11100110)
  XOR:     0x01 (00000001)
  descripcion: flip de bit dentro de la firma Ed25519 (offset 172)
```

```
--- INTENTO DE DESCIFRADO / VERIFICACION ---
```

```
llamada: verify_container(tampered_signed, alice_sign_pub)
```

```
[EXCEPCION CAPTURADA]
```

```
tipo..... cryptography.exceptions.InvalidSignature
```

```
traceback completo:
```

```
File "crypto/signatures.py", line 143, in verify_container
    public_key.verify(signature, data_to_verify)
cryptography.exceptions.InvalidSignature
```

```
[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext
```

¿El sistema detectó la modificación? Sí, sin problema. La verificación falla apenas pasamos el archivo por `verify_container`. Y eso es importante porque nuestro sistema verifica la firma ANTES de intentar

descifrar, así que cuando algo está raro ni se gasta tiempo procesando datos posiblemente forjados.

Propiedad de seguridad afectada. *Autenticidad.* Esto sirve para confirmar quién firmó el archivo. Si alguien intenta cambiar la firma, el sistema se da cuenta inmediatamente. Ed25519 es un algoritmo bastante robusto y básicamente no se puede falsificar una firma sin tener la llave privada del firmante.

3.5 Modificación de Key Identifier

Descripción. En los archivos firmados de la versión completa del sistema (D5), aparte de la firma misma se guarda un identificador del firmante (un fingerprint de 32 bytes) que dice “esto lo firmó Alice”. Lo que queríamos ver aquí es qué pasa si un atacante intenta cambiar ese identificador para hacerse pasar por otra persona, dejando la firma original intacta.

Qué se modificó. Cambiamos los 32 bytes del fingerprint que dice “Alice” por el fingerprint de otra persona, que en este caso le pusimos Eve. La idea es simular un ataque donde alguien quiere robar la firma de Alice y atribuírsela a otro.

Comportamiento observado. El sistema lo detectó al instante y nos lanzó `InvalidSignature` con un mensaje específico que dice que el fingerprint del archivo no coincide con la llave que le pasamos para verificar. La salida completa:

```
=====
TEST 5: Modificacion de KEY IDENTIFIER (signer fingerprint en footer)
=====

--- FASE 1 - GENERACION DE LLAVES Y CONTENEDOR D5 FIRMADO ---
Alice fingerprint..... 667eac326624a74e8aa10b7aa12e6f90f9f450c5d99d0fbfc4a3f0c5101a86c3
Eve  fingerprint..... cda20c19e4e61843f9a1f7baf9ff5b23239f79626287d98df8013026a5e15d5e
contenedor D5..... 323 bytes
layout final..... SDDH(...) || SIGS(4) || SIGNER_FP(32) || SIG(64)
SIGNER_FP en footer..... 667eac326624a74e8aa10b7aa12e6f90f9f450c5d99d0fbfc4a3f0c5101a86c3
coincide con Alice?..... True

--- FASE 2 - VERIFICACION DEL HAPPY PATH ---
plaintext recuperado..... b'Mensaje secreto firmado por Alice'
[OK] firma valida + Bob descifra

--- FASE 3 - APLICAR MODIFICACION: sustituir SIGNER_FP de Alice por el de Eve ---
bytes[227:259] = SIGNER_FP
    antes:  667eac326624a74e8aa10b7aa12e6f90f9f450c5d99d0fbfc4a3f0c5101a86c3
    despues: cda20c19e4e61843f9a1f7baf9ff5b23239f79626287d98df8013026a5e15d5e

--- INTENTO DE DESCIFRADO / VERIFICACION ---
llamada: secure_verify_and_decrypt(tampered, alice_sign_pub, bob_priv)

[EXCEPCION CAPTURADA]
tipo..... cryptography.exceptions.InvalidSignature
mensaje..... 'El fingerprint del contenedor no coincide con la
               llave publica proporcionada'

traceback completo:
  File "crypto/secure_send.py", line 143, in secure_verify_and_decrypt
    sddh_clean = verify_hybrid_container(signed_container, expected_signer_pub)
  File "crypto/signatures.py", line 136, in verify_container
    raise InvalidSignature(
cryptography.exceptions.InvalidSignature: El fingerprint del contenedor no
coincide con la llave publica proporcionada
```

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

¿El sistema detectó la modificación? Sí, y de hecho con un mensaje bien claro: “el fingerprint del contenedor no coincide con la llave pública proporcionada”. O sea, el sistema ve que el archivo dice “esto lo firmó Eve” pero le estamos pasando la llave de Alice, y de inmediato sabe que algo está raro. Aparte de eso, aunque ese chequeo no estuviera, la firma original se hizo incluyendo el fingerprint de Alice, así que cualquier cambio igual rompería la verificación.

Propiedad de seguridad afectada. *Autenticidad e identidad del firmante.* Aquí lo que se protege es que alguien no pueda quedarse con la firma de otro y atribuírsela a una tercera persona. El sistema “amarra” la identidad del firmante a la firma misma, así que no puedes cambiar uno sin romper el otro.

4. Evidencia visual

Esta es una captura de cómo se ve el script cuando lo corremos en la terminal. Sale con todos los detalles del proceso: el archivo en hexadecimal antes y después de la modificación, la verificación del happy path, y la excepción al final. La captura es del Test 1 (modificación de metadata) pero los demás tests salen con el mismo formato.

```
+ Proyecto git:(feature/d3-hybrid) * clear
+ Proyecto git:(feature/d3-hybrid) * python3 audit_tampering.py --no-color | tee docs/audit_log.txt

#####
# AUDITORIA DE SEGURIDAD DEL SDDV - 5 VECTORES DE MODIFICACION #
#####

El SDDV (Secure Digital Document Vault) implementa cifrado AEAD,
cifrado híbrido multi-destinatario y firma Ed25519 sobre el contenido
completo. Esta auditoria modifica byte-a-byte cada componente critico
y verifica que el sistema lo detecta antes de exponer plaintext.

=====
TEST 1: Modificacion de METADATA (filename en AAD del SDDV)
=====

--- FASE 1 - GENERACION DE CONTENEDOR LIMPIO ---
plaintext..... b'Documento confidencial de auditoria'
plaintext (hex)..... 446f63756d656e746f20636f6e666964656e6369616c2064652061756469746f726961
filename..... 'contrato.pdf'
algoritmo..... AES-256-GCM
llave generada..... cf0bca6e21afc44727b0125e819d664c543d73671204d8efbc9f8c651d43f8bc
contenedor (bytes)..... 95

hex dump (contenedor SDDV original, 95 bytes total):
00000000 53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 0c |SDDV.....i.S...|
00000010 63 6f 6e 74 72 61 74 6f 2e 70 64 66 67 e5 14 62 |bontrato.pdfg..b|
00000020 ca 83 6f 0d 41 33 32 30 00 00 00 23 3c f0 ad 74 |..o.A320...#<..t|
00000030 1f eb 7f d8 bf 26 25 43 96 dc e0 bb ae 85 c2 e8 |....&%C.....|
00000040 2b 69 48 bc e0 6e c2 95 0c 4b 71 a2 bb 71 ac 22 |+iH..n...Kq..q."|
00000050 c5 6b 57 8d 08 4b ea 56 37 d5 07 02 5c 00 43 |.kW..K.V7...\C|

--- FASE 2 - VERIFICACION DEL HAPPY PATH ---
primero confirmamos que el contenedor LIMPIO se descifra bien:
plaintext recuperado..... b'Documento confidencial de auditoria'
metadata recuperada..... {'version': 1, 'algo': <Algorithm.AES_256_GCM: 1>, 'timestamp': 1778012936, 'filename': 'contrato.pdf'}
[OK] happy path funciona correctamente

--- FASE 3 - APLICAR MODIFICACION MALICIOSA ---

>> MODIFICACION en byte[16]:
antes: 0x63 (01100011) 'c'
despues: 0x62 (01100010) 'b'
XOR: 0x01 (00000001)
descripcion: flip de bit en filename (offset 16 = primer caracter de 'contrato.pdf')

hex dump (contenedor SDDV ADULTERADO, 95 bytes total):
00000000 53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 0c |SDDV.....i.S...|
00000010 62 6f 6e 74 72 61 74 6f 2e 70 64 66 67 e5 14 62 |bontrato.pdfg..b|
00000020 ca 83 6f 0d 41 33 32 30 00 00 00 23 3c f0 ad 74 |..o.A320...#<..t|
00000030 1f eb 7f d8 bf 26 25 43 96 dc e0 bb ae 85 c2 e8 |....&%C.....|
00000040 2b 69 48 bc e0 6e c2 95 0c 4b 71 a2 bb 71 ac 22 |+iH..n...Kq..q."|
00000050 c5 6b 57 8d 08 4b ea 56 37 d5 07 02 5c 00 43 |.kW..K.V7...\C|

--- FASE 4 - INTERPRETACION DEL ATAQUE ---
el filename forma parte del AAD del cifrado AES-256-GCM.
modificarlo cambia el AAD presentado al verificar el TAG.
el TAG calculado durante encrypt cubre 'AAD original + ciphertext'.
al verificar con AAD modificado, el TAG no coincide -> InvalidTag.

--- INTENTO DE DESCIFRADO / VERIFICACION ---
llamada: decrypt_file(tampered_container, key)
ejecutando...

[EXCEPCION CAPTURADA]
tipo..... cryptography.exceptions.InvalidTag
mensaje..... <vacío>
```

Figure 1: Captura de la ejecución del Test 1 (modificación de metadata) con el comando `python3 audit_tampering.py --no-color | tee docs/audit_log.txt`

5. Resumen de resultados

Esta es la tabla con los resultados de las cinco pruebas que hicimos:

Prueba	Componente	Propiedad	Detectado
Modificación de Metadata	aead.py	Integridad	Sí — InvalidTag
Lista de Destinatarios	hybrid.py	Integridad/Acceso	Sí — ValueError
Modificación de Nonce	aead.py	Conf./Integridad	Sí — InvalidTag
Firma Digital Ed25519	signatures.py	Autenticidad	Sí — InvalidSignature
Key Identifier (SIGNER_FP)	signatures.py	Autenticidad/Identidad	Sí — InvalidSignature

6. Conclusión

Después de hacer las cinco pruebas, podemos decir que el sistema SDDV sí detecta correctamente cuando alguien modifica algo del archivo cifrado, sin importar qué parte se modifique. En todos los casos el sistema nos rechazó el archivo modificado y nos lanzó una excepción explicando más o menos qué fue lo que falló.

Una cosa que nos pareció interesante es que el sistema sigue lo que se llama el principio “fail-closed”, que básicamente significa que cuando algo no cuadra, prefiere no devolver nada en lugar de devolver datos posiblemente comprometidos. Eso quiere decir que ni un solo byte del contenido original se ve cuando intentas abrir un archivo manipulado. Eso nos parece una buena decisión de diseño.

También nos sirvió para entender por qué el sistema verifica primero la firma antes de intentar descifrar (el patrón verify-first): si lo hiciera al revés, gastaría recursos descifrando algo que después iba a rechazar de todos modos, y además podría exponerse a otros ataques. Hacerlo en el orden correcto cierra esos huecos.

Lo que nos quedó claro de esta auditoría es que las tres capas de protección que tiene el sistema (cifrado AEAD, cifrado híbrido para varios destinatarios y firma digital) trabajan en conjunto y se complementan. Si una falla, las otras siguen detectando los ataques.

Anexo A — Script de auditoría

El script `audit_tampering.py` está en la raíz del repositorio y hace todo el proceso de los cinco ataques de manera automática. Para cada uno hace lo siguiente:

1. Genera llaves nuevas y un archivo cifrado limpio.
2. Verifica que el archivo limpio se pueda descifrar bien (eso es el “happy path”) como prueba de control.
3. Le aplica la modificación maliciosa al archivo, byte por byte.
4. Imprime el archivo en hexadecimal antes y después, marcando el byte que se cambió.
5. Intenta descifrar o verificar el archivo modificado y captura la excepción que lance el sistema.

Para correr el script y guardar el log:

```
python3 audit_tampering.py --no-color | tee docs/audit_log.txt
```

Anexo B — Log completo de ejecución

Aquí va la salida completa del script con los cinco tests, tal cual sale en la terminal. Cada test tiene la generación de llaves y archivo, los hex dumps, la verificación del happy path, la modificación que aplicamos, la interpretación del ataque y el traceback completo de la excepción.

```
#####  
#          AUDITORIA DE SEGURIDAD DEL SDDV - 5 VECTORES DE MODIFICACION          #  
#####
```

El SDDV (Secure Digital Document Vault) implementa cifrado AEAD, cifrado hibrido multi-destinatario y firma Ed25519 sobre el contenedor completo. Esta auditoria modifica byte-a-byte cada componente critico y verifica que el sistema lo detecta antes de exponer plaintext.

```
=====
```

TEST 1: Modificacion de METADATA (filename en AAD del SDDV)

```
=====
```

--- FASE 1 - GENERACION DE CONTENEDOR LIMPIO ---

plaintext..... b'Documento confidencial de auditoria'

plaintext (hex)..... 446f63756d656e746f20636f6e666964656e6369616c2064652061756469746f

filename..... 'contrato.pdf'

algoritmo..... AES-256-GCM

llave generada..... cf0bca6e21afc44727b0125e819d664c543d73671204d8efbc9f8c651d43f8bc

contenedor (bytes)..... 95

hex dump (contenedor SDDV original, 95 bytes total):

00000000	53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 0c	SDDV.....i.S...
00000010	63 6f 6e 74 72 61 74 6f 2e 70 64 66 67 e5 14 62	contrato.pdfg..b
00000020	ca 83 6f 0d 41 33 32 30 00 00 00 23 3c f0 ad 74	..o.A320...#<..t
00000030	1f eb 7f d8 bf 26 25 43 96 dc e0 bb ae 85 c2 e8	&%C.....
00000040	2b 69 48 bc e0 6e c2 95 0c 4b 71 a2 bb 71 ac 22	+iH..n...Kq..q."
00000050	c5 6b 57 8d 08 4b ea 56 37 d5 07 02 5c 00 43	.kW..K.V7...\C

--- FASE 2 - VERIFICACION DEL HAPPY PATH ---

primero confirmamos que el contenedor LIMPIO se descifra bien:

plaintext recuperado..... b'Documento confidencial de auditoria'

metadata recuperada..... {'version': 1, 'algo': <Algorithm.AES_256_GCM: 1>, 'timestamp':

[OK] happy path funciona correctamente

--- FASE 3 - APLICAR MODIFICACION MALICIOSA ---

>> MODIFICACION en byte[16]:

antes: 0x63 (01100011) 'c'

despues: 0x62 (01100010) 'b'

XOR: 0x01 (00000001)

descripcion: flip de bit en filename (offset 16 = primer caracter de 'contrato.pdf')

```

hex dump (contenedor SDDV ADULTERADO, 95 bytes total):
00000000  53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 0c |SDDV.....i.S...|
00000010  62 6f 6e 74 72 61 74 6f 2e 70 64 66 67 e5 14 62 |bontrato.pdfg..b|
00000020  ca 83 6f 0d 41 33 32 30 00 00 00 23 3c f0 ad 74 |...o.A320...#<..t|
00000030  1f eb 7f d8 bf 26 25 43 96 dc e0 bb ae 85 c2 e8 |.....&%C.....|
00000040  2b 69 48 bc e0 6e c2 95 0c 4b 71 a2 bb 71 ac 22 |+iH..n...Kq..q."|
00000050  c5 6b 57 8d 08 4b ea 56 37 d5 07 02 5c 00 43   |.kW..K.V7...\C|

```

--- FASE 4 - INTERPRETACION DEL ATAQUE ---

el filename forma parte del AAD del cifrado AES-256-GCM.
 modificarlo cambia el AAD presentado al verificar el TAG.
 el TAG calculado durante encrypt cubre 'AAD original + ciphertext'.
 al verificar con AAD modificado, el TAG no coincide -> InvalidTag.

--- INTENTO DE DESCIFRADO / VERIFICACION ---

llamada: decrypt_file(tampered_container, key)
 ejecutando...

[EXCEPCION CAPTURADA]

tipo..... cryptography.exceptions.InvalidTag
 mensaje..... <vacio>

traceback completo:

Traceback (most recent call last):

```

File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    result = fn()
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    lambda: decrypt_file(bytes(tampered), key),
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/aead.py", line 192
    plaintext = cipher.decrypt(nonce, ciphertext + tag, header)
cryptography.exceptions.InvalidTag

```

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

=====

TEST 2: Modificacion de LISTA DE DESTINATARIOS (fingerprint en AAD)

=====

--- FASE 1 - GENERACION DE LLAVES Y CONTENEDOR HIBRIDO ---

```

Alice X25519 FP..... 94b9eea73e4ea73f27d61e5e9b84662ff7273e0bae613aa2f037a7869712c647
Bob   X25519 FP..... f96fa7f343de01c73c5feda83819edd576c6e3baaaf77057f2b7644e7c5585f1
plaintext..... b'Documento compartido entre Alice y Bob'
filename..... 'doc.pdf'
contenedor (bytes)..... 343
destinatarios..... 2

```

hex dump (contenedor SDDH original, 343 bytes total):

```

00000000  53 44 44 48 01 01 00 00 00 00 69 fa 53 08 00 07 |SDDH.....i.S...|
00000010  64 6f 63 2e 70 64 66 00 02 94 b9 ee a7 3e 4e a7 |doc.pdf.....>N.|
00000020  3f 27 d6 1e 5e 9b 84 66 2f f7 27 3e 0b ae 61 3a |?'...^..f/.'>..a:|

```



```

00000030 a2 f0 37 a7 86 97 12 c6 47 76 7d 61 62 89 cc c7 |..7.....Gv}ab...|
00000040 a5 a5 27 88 2e 32 57 1b b5 40 17 8e 9d b5 ba 19 |..'..2W...@.....|
00000050 25 a9 a6 eb bd ba 85 1d 64 d2 c4 96 42 42 6a ca |%.....d...BBj.|
00000060 c6 4d 69 08 36 c2 c1 70 ec 86 a8 00 22 3b d9 0f |.Mi.6..p....";...|
00000070 b1 95 73 ad 8c 45 3e 41 ff 33 80 3c 2c 7b 40 de |..s..E>A.3.<,{@.|
00000080 56 35 ac b3 f2 da 5d c0 15 bd f3 d2 3c 08 cf 24 |V5....].....<...$|
00000090 35 ff c7 10 21 f9 6f a7 f3 43 de 01 c7 3c 5f ed |5...!.o..C...<_.|
... (183 bytes mas)

```

--- FASE 2 - VERIFICACION DEL HAPPY PATH (Alice y Bob descifran) ---

Alice recupera..... b'Documento compartido entre Alice y Bob'

Bob recupera..... b'Documento compartido entre Alice y Bob'

[OK] ambos destinatarios descifran correctamente

--- FASE 3 - APLICAR MODIFICACION MALICIOSA ---

>> MODIFICACION en byte[25]:

antes: 0x94 (10010100) '.'

despues: 0x95 (10010101) '.'

XOR: 0x01 (00000001)

descripcion: flip de bit en primer byte del fingerprint de Alice (offset 25)

hex dump (contenedor SDDH ADULTERADO, 343 bytes total):

```

00000000 53 44 44 48 01 01 00 00 00 00 69 fa 53 08 00 07 |SDDH.....i.S...|
00000010 64 6f 63 2e 70 64 66 00 02 95 b9 ee a7 3e 4e a7 |doc.pdf.....>N.|
00000020 3f 27 d6 1e 5e 9b 84 66 2f f7 27 3e 0b ae 61 3a |?'..^..f/. '>..a:|
00000030 a2 f0 37 a7 86 97 12 c6 47 76 7d 61 62 89 cc c7 |..7.....Gv}ab...|
00000040 a5 a5 27 88 2e 32 57 1b b5 40 17 8e 9d b5 ba 19 |..'..2W...@.....|
00000050 25 a9 a6 eb bd ba 85 1d 64 d2 c4 96 42 42 6a ca |%.....d...BBj.|
00000060 c6 4d 69 08 36 c2 c1 70 ec 86 a8 00 22 3b d9 0f |.Mi.6..p....";...|
00000070 b1 95 73 ad 8c 45 3e 41 ff 33 80 3c 2c 7b 40 de |..s..E>A.3.<,{@.|
00000080 56 35 ac b3 f2 da 5d c0 15 bd f3 d2 3c 08 cf 24 |V5....].....<...$|
00000090 35 ff c7 10 21 f9 6f a7 f3 43 de 01 c7 3c 5f ed |5...!.o..C...<_.|
... (183 bytes mas)

```

--- FASE 4 - INTERPRETACION DEL ATAQUE ---

el fingerprint de Alice ya no coincide con SHA-256(alice_pub).

cuando Alice intente descifrar, decrypt_for_recipient calcula su fingerprint y lo busca en la lista. NO encuentra coincidencia.

-> ValueError 'destinatario no autorizado' antes de tocar el ciphertext.

ademas, la lista es parte del AAD: si forzaramos seguir, el TAG fallaria.

--- INTENTO DE DESCIFRADO / VERIFICACION ---

llamada: decrypt_for_recipient(tampered, alice_priv)

ejecutando...

[EXCEPCION CAPTURADA]

tipo..... builtins.ValueError

mensaje..... 'Este destinatario no esta autorizado en el contenedor'

traceback completo:

Traceback (most recent call last):

File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
result = fn()

File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
lambda: decrypt_for_recipient(bytes(tampered), alice_priv),

File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/hybrid.py", line 3
raise ValueError("Este destinatario no esta autorizado en el contenedor")

ValueError: Este destinatario no esta autorizado en el contenedor

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

=====

TEST 3: Modificacion de NONCE (DEM nonce de 96 bits)

=====

--- FASE 1 - GENERACION DE CONTENEDOR LIMPIO ---

plaintext..... b'Mensaje protegido por AES-256-GCM'

llave..... d6a8bc0306e7d6434d1d33dc0e7e17c4e65f0fc679e6ea9c8851e5d64a26cb4e

contenedor bytes..... 88

nonce offset..... 23

nonce original..... 71e9580c4bfe5584886bda28

hex dump (contenedor SDDV original, 88 bytes total):

00000000	53 44 44 56 01 01 00 00	00 00 69 fa 53 08 00 07	SDDV.....i.S...
00000010	64 6f 63 2e 74 78 74 71	e9 58 0c 4b fe 55 84 88	doc.txtq.X.K.U..
00000020	6b da 28 00 00 00 21 94	75 aa ef 01 df 87 32 34	k.(...!.u.....24
00000030	5c 96 4a 41 14 31 06 18	7d a0 ec 31 37 b2 da df	\.JA.1..}..17...
00000040	97 9e 33 7d 05 ad 4a 17	2e 1b d6 10 42 72 d2 df	\.3}..J.....Br..
00000050	f8 89 fd 88 c6 71 f2 b1		q..

(los bytes resaltados son los 12 bytes del nonce)

--- FASE 2 - APLICAR MODIFICACION AL NONCE ---

>> MODIFICACION en byte[23]:

antes: 0x71 (01110001) 'q'

despues: 0x8e (10001110) '.'

XOR: 0xff (11111111)

descripcion: flip total (XOR 0xFF) del primer byte del nonce

nonce modificado..... 8ee9580c4bfe5584886bda28

hex dump (contenedor SDDV ADULTERADO, 88 bytes total):

00000000	53 44 44 56 01 01 00 00	00 00 69 fa 53 08 00 07	SDDV.....i.S...
00000010	64 6f 63 2e 74 78 74 8e	e9 58 0c 4b fe 55 84 88	doc.txt..X.K.U..
00000020	6b da 28 00 00 00 21 94	75 aa ef 01 df 87 32 34	k.(...!.u.....24
00000030	5c 96 4a 41 14 31 06 18	7d a0 ec 31 37 b2 da df	\.JA.1..}..17...
00000040	97 9e 33 7d 05 ad 4a 17	2e 1b d6 10 42 72 d2 df	\.3}..J.....Br..
00000050	f8 89 fd 88 c6 71 f2 b1		q..

--- FASE 3 - INTERPRETACION DEL ATAQUE ---

AES-GCM usa el nonce para derivar el counter $J_0 = \text{nonce} \parallel 0x00000001$.
 el keystream es AES-CTR(K, J_0+1 , J_0+2 , ...) y el TAG = GHASH(H, AAD, ciphertext) XOR AES_K(J_0). modificar el nonce cambia ambos: keystream
 Y tag. el descifrado no recupera plaintext y la verificacion falla.

```
--- INTENTO DE DESCIFRADO / VERIFICACION ---
llamada: decrypt_file(tampered_container, key)
ejecutando...
```

```
[EXCEPCION CAPTURADA]
tipo..... cryptography.exceptions.InvalidTag
mensaje..... <vacio>
```

```
traceback completo:
Traceback (most recent call last):
  File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    result = fn()
  File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    lambda: decrypt_file(bytes(tampered), key),
  File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/aead.py", line 192
    plaintext = cipher.decrypt(nonce, ciphertext + tag, header)
cryptography.exceptions.InvalidTag
```

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

=====

TEST 4: Modificacion de FIRMA DIGITAL (Ed25519)

=====

--- FASE 1 - GENERACION DE LLAVES Y CONTENEDOR FIRMADO ---

```
Alice Ed25519 pub..... 64c6a8867d3206d6d99870d3eb3be5436d04a1246e6850f50dfd20563beac3f0
Alice fingerprint..... 32177b9c961e0abf4f01ac55b49f4fea2d7036311943160bda73995c927fb731
contenedor SDDV..... 82 bytes
contenedor firmado..... 182 bytes (= SDDV + 100 bytes footer)
layout footer..... SIGS(4) + FINGERPRINT(32) + SIGNATURE(64)
firma Ed25519 (hex)..... bebe1e9aa788a3f5e037273d34e045311ddcc6598a818791efb0aa046e1176b7
```

hex dump (contenedor firmado completo, 182 bytes total):

```
00000000 53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 07 |SDDV.....i.S...|
00000010 64 6f 63 2e 70 64 66 bb 55 a9 83 0e ce 39 c1 81 |doc.pdf.U....9..|
00000020 f2 4b 34 00 00 00 1b 2c 60 0f 61 79 84 49 6b 37 |.K4....,.ay.Ik7|
00000030 89 9f 87 e6 b3 0b c7 e1 73 ab e8 6a f8 dd d0 80 |.....s..j....|
00000040 b6 10 eb c6 a7 83 c9 8c 65 ca a5 cd ae f4 5f 2a |.....e....._*|
00000050 96 41 53 49 47 53 32 17 7b 9c 96 1e 0a bf 4f 01 |.ASIGS2.{.....0.|
00000060 ac 55 b4 9f 4f ea 2d 70 36 31 19 43 16 0b da 73 |.U..0.-p61.C...s|
00000070 99 5c 92 7f b7 31 be be 1e 9a a7 88 a3 f5 e0 37 |.\...1.....7|
00000080 27 3d 34 e0 45 31 1d dc c6 59 8a 81 87 91 ef b0 |'=4.E1...Y.....|
00000090 aa 04 6e 11 76 b7 4b b7 ee 03 a1 16 c6 1d 11 2f |..n.v.K...../|
000000a0 ac fa 65 be 41 3f 6c e7 26 e2 c7 7f e7 ce 22 62 |..e.A?1.&....."b|
000000b0 9f 05 ad ca 74 0c                                |....t.|
```

(los 64 bytes resaltados son la firma Ed25519)

--- FASE 2 - VERIFICACION DEL HAPPY PATH ---

```
verify_container retorno 82 bytes (= SDDV original)
plaintext recuperado..... b'Documento firmado por Alice'
[OK] firma valida + descifrado correcto
```

--- FASE 3 - APLICAR MODIFICACION A LA FIRMA ---

```
>> MODIFICACION en byte[172]:
    antes:  0xe7 (11100111)  '.'
    despues: 0xe6 (11100110)  '.'
    XOR:     0x01 (00000001)
    descripcion: flip de bit dentro de la firma Ed25519 (offset 172)
```

hex dump (contenedor firmado ADULTERADO, 182 bytes total):

```
00000000  53 44 44 56 01 01 00 00 00 00 69 fa 53 08 00 07 |SDDV.....i.S...|
00000010  64 6f 63 2e 70 64 66 bb 55 a9 83 0e ce 39 c1 81 |doc.pdf.U....9..|
00000020  f2 4b 34 00 00 00 1b 2c 60 0f 61 79 84 49 6b 37 |.K4....,`.ay.Ik7|
00000030  89 9f 87 e6 b3 0b c7 e1 73 ab e8 6a f8 dd d0 80 |.....s..j....|
00000040  b6 10 eb c6 a7 83 c9 8c 65 ca a5 cd ae f4 5f 2a |.....e....._*|
00000050  96 41 53 49 47 53 32 17 7b 9c 96 1e 0a bf 4f 01 |.ASIGS2.{.....0.|
00000060  ac 55 b4 9f 4f ea 2d 70 36 31 19 43 16 0b da 73 |.U..0.-p61.C...s|
00000070  99 5c 92 7f b7 31 be be 1e 9a a7 88 a3 f5 e0 37 |.\...1.....7|
00000080  27 3d 34 e0 45 31 1d dc c6 59 8a 81 87 91 ef b0 |'=4.E1...Y.....|
00000090  aa 04 6e 11 76 b7 4b b7 ee 03 a1 16 c6 1d 11 2f |..n.v.K...../|
000000a0  ac fa 65 be 41 3f 6c e7 26 e2 c7 7f e6 ce 22 62 |...e.A?l.&....."b|
000000b0  9f 05 ad ca 74 0c                                |....t.|
```

--- FASE 4 - INTERPRETACION DEL ATAQUE ---

Ed25519 calcula la firma como (R, S) donde $S = \text{hash}(R, A, M) * a + r$. cualquier flip de bit produce una (R, S) que no satisface la ecuacion de verificacion: $[S]B == R + [\text{hash}(R, A, M)]A$. la libreria detecta esto y lanza InvalidSignature. Ed25519 es EUF-CMA seguro -> imposible forjar.

--- INTENTO DE DESCIFRADO / VERIFICACION ---

```
llamada: verify_container(tampered_signed, alice_sign_pub)
ejecutando...
```

[EXCEPCION CAPTURADA]

```
tipo..... cryptography.exceptions.InvalidSignature
mensaje..... <vacio>
```

traceback completo:

Traceback (most recent call last):

```
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    result = fn()
```

```
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    lambda: verify_container(bytes(tampered), alice_sign_pub),
```

```
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/signatures.py", li
```

```
public_key.verify(signature, data_to_verify)
cryptography.exceptions.InvalidSignature
```

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

```
=====
TEST 5: Modificacion de KEY IDENTIFIER (signer fingerprint en footer)
=====
```

--- FASE 1 - GENERACION DE LLAVES Y CONTENEDOR D5 FIRMADO ---

```
Alice fingerprint..... 667eac326624a74e8aa10b7aa12e6f90f9f450c5d99d0fbfc4a3f0c5101a86c3
Eve  fingerprint..... cda20c19e4e61843f9a1f7baf9ff5b23239f79626287d98df8013026a5e15d5e
contenedor D5..... 323 bytes
layout final..... SDDH(...) || SIGS(4) || SIGNER_FP(32) || SIG(64)
SIGNER_FP en footer..... 667eac326624a74e8aa10b7aa12e6f90f9f450c5d99d0fbfc4a3f0c5101a86c3
coincide con Alice?..... True
```

hex dump (contenedor D5 original, 323 bytes total):

```
00000000 53 44 44 48 01 01 00 00 00 00 69 fa 53 08 00 10 |SDDH.....i.S...|
00000010 63 6f 6e 66 69 64 65 6e 63 69 61 6c 2e 70 64 66 |confidencial.pdf|
00000020 00 01 bd 57 9c a0 60 83 da ee 4a ea 10 85 0a d9 |...W..`...J.....|
00000030 75 18 aa 97 10 f7 92 a4 4f f4 fe a2 63 42 e4 0e |u.....0...cB...|
00000040 9a 5f b0 73 96 bd f4 c7 db 5f 0d 59 d4 90 15 c7 |...s....._Y....|
00000050 1c 66 73 f5 d3 b1 bf cb 38 1a 69 eb 33 2c d8 d1 |.fs.....8.i.3...|
00000060 6e 1b d7 01 83 c8 dc c2 2f 54 ba 00 67 e7 3b c4 |n...../T..g.;..|
00000070 9a 82 03 d7 5f 86 08 56 57 23 60 ad da 09 59 9b |...._..VW#`...Y..|
00000080 54 d5 e2 04 5b 0b ea a8 47 19 b0 8b cb 1b fd e5 |T...[...G.....|
00000090 2b b5 a5 20 e8 94 ba fb 23 ad 9b 6e 20 9d 9a 18 |+.. ....#..n ...|
000000a0 c4 15 a1 15 0b 4e 1b 75 ee 5b 00 00 00 21 a2 e4 |....N.u.[...!...|
000000b0 89 59 28 ce 94 92 91 a4 e4 40 d5 1f 98 46 2b 6b |.Y(.....@...F+k|
000000c0 b1 b2 49 a3 8d 92 05 1a a7 b6 f8 36 16 bf f8 88 |..I.....6....|
000000d0 2a 4e 6a ad 7c f5 cb 85 c9 da ec 2a 5e 8b f9 53 |*Nj.|.....*~..S|
000000e0 49 47 53 66 7e ac 32 66 24 a7 4e 8a a1 0b 7a a1 |IGSf~.2f$.N...z..|
000000f0 2e 6f 90 f9 f4 50 c5 d9 9d 0f bf c4 a3 f0 c5 10 |.o...P.....|
00000100 1a 86 c3 0f 3c 6f 9a 9c fc 7c c7 0a 56 96 b9 06 |....<o...|.V...|
00000110 77 26 b8 54 97 6c 78 d8 b1 79 cf 9f c6 7e 04 3f |w&.T.lx..y...~.?|
00000120 ec 92 79 e0 f3 fa 7e 8e 49 c7 6c a1 99 26 86 db |..y...~.I.l..&..|
00000130 a1 32 01 8f c6 d5 c8 88 7e 06 dd d9 b1 b8 00 87 |.2.....~.....|
... (3 bytes mas)
```

(los 32 bytes resaltados son el SIGNER_FP)

--- FASE 2 - VERIFICACION DEL HAPPY PATH ---

```
plaintext recuperado..... b'Mensaje secreto firmado por Alice'
[OK] firma valida + Bob descifra
```

--- FASE 3 - APLICAR MODIFICACION: sustituir SIGNER_FP de Alice por el de Eve ---

```
bytes[227:259] = SIGNER_FP
```

```
antes: 667eac326624a74e8aa10b7aa12e6f90f9f450c5d99d0fbfc4a3f0c5101a86c3
despues: cda20c19e4e61843f9a1f7baf9ff5b23239f79626287d98df8013026a5e15d5e
```

```

hex dump (contenedor D5 ADULTERADO, 323 bytes total):
00000000  53 44 44 48 01 01 00 00 00 00 69 fa 53 08 00 10 |SDDH.....i.S...|
00000010  63 6f 6e 66 69 64 65 6e 63 69 61 6c 2e 70 64 66 |confidencial.pdf|
00000020  00 01 bd 57 9c a0 60 83 da ee 4a ea 10 85 0a d9 |...W..`...J.....|
00000030  75 18 aa 97 10 f7 92 a4 4f f4 fe a2 63 42 e4 0e |u.....0...cB..|
00000040  9a 5f b0 73 96 bd f4 c7 db 5f 0d 59 d4 90 15 c7 |_.s....._Y....|
00000050  1c 66 73 f5 d3 b1 bf cb 38 1a 69 eb 33 2c d8 d1 |.fs.....8.i.3,..|
00000060  6e 1b d7 01 83 c8 dc c2 2f 54 ba 00 67 e7 3b c4 |n...../T..g.;..|
00000070  9a 82 03 d7 5f 86 08 56 57 23 60 ad da 09 59 9b |...._..VW#`...Y.|
00000080  54 d5 e2 04 5b 0b ea a8 47 19 b0 8b cb 1b fd e5 |T...[...G.....|
00000090  2b b5 a5 20 e8 94 ba fb 23 ad 9b 6e 20 9d 9a 18 |+.. ....#..n ...|
000000a0  c4 15 a1 15 0b 4e 1b 75 ee 5b 00 00 00 21 a2 e4 |.....N.u.[...!..|
000000b0  89 59 28 ce 94 92 91 a4 e4 40 d5 1f 98 46 2b 6b |.Y(.....@...F+k|
000000c0  b1 b2 49 a3 8d 92 05 1a a7 b6 f8 36 16 bf f8 88 |..I.....6....|
000000d0  2a 4e 6a ad 7c f5 cb 85 c9 da ec 2a 5e 8b f9 53 |*Nj.|.....*^..S|
000000e0  49 47 53 cd a2 0c 19 e4 e6 18 43 f9 a1 f7 ba f9 |IGS.....C.....|
000000f0  ff 5b 23 23 9f 79 62 62 87 d9 8d f8 01 30 26 a5 |. [##.ybb.....0&.|
00000100  e1 5d 5e 0f 3c 6f 9a 9c fc 7c c7 0a 56 96 b9 06 |.]^.<o...|.V...|
00000110  77 26 b8 54 97 6c 78 d8 b1 79 cf 9f c6 7e 04 3f |w&.T.lx..y...~.?|
00000120  ec 92 79 e0 f3 fa 7e 8e 49 c7 6c a1 99 26 86 db |..y...~.I.l..&..|
00000130  a1 32 01 8f c6 d5 c8 88 7e 06 dd d9 b1 b8 00 87 |.2.....~.....|
... (3 bytes mas)

```

--- FASE 4 - INTERPRETACION DEL ATAQUE ---

el atacante intenta hacerse pasar por Eve manteniendo la firma de Alice.
 pero verify_container compara fingerprint con SHA-256(expected_pub).
 con expected=alice_sign_pub: el FP del footer (Eve) != SHA256(Alice).
 -> InvalidSignature antes de verificar la firma Ed25519 misma.
 ademas, la firma fue calculada sobre 'SDDH || SIGS || alice_fp', asi
 que aunque pasara ese chequeo, Ed25519.verify(sig, ... eve_fp) fallaria.

--- INTENTO DE DESCIFRADO / VERIFICACION ---

llamada: secure_verify_and_decrypt(tampered, alice_sign_pub, bob_priv)
 ejecutando...

[EXCEPCION CAPTURADA]

tipo..... cryptography.exceptions.InvalidSignature
 mensaje..... 'El fingerprint del contenedor no coincide con la llave publica

traceback completo:

Traceback (most recent call last):

```

File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    result = fn()
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/audit_tampering.py", line
    lambda: secure_verify_and_decrypt(bytes(tampered), alice_sign_pub, bob_priv),
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/secure_send.py", line
    sddh_clean = verify_hybrid_container(signed_container, expected_signer_pub)
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/signatures.py", line
    return verify_container(signed_sddh, signer_pub)
File "/Users/emiliocontreras/Documents/10Semestre/Cripto/Proyecto/crypto/signatures.py", line

```

```
raise InvalidSignature(  
    cryptography.exceptions.InvalidSignature: El fingerprint del contenedor no coincide con la ll
```

[OK] sistema fail-closed: lanza excepcion antes de exponer plaintext

```
#####  
#                               #  
#               RESUMEN DE RESULTADOS               #  
#####
```

TEST	DETECTADO?	PROPIEDAD
-----	-----	-----
Metadata	[OK] SI	Integridad
Lista destinatarios	[OK] SI	Integridad/Acceso
Nonce	[OK] SI	Conf./Integridad
Firma Ed25519	[OK] SI	Autenticidad
Key identifier	[OK] SI	Autenticidad/Identidad

CONCLUSION: el sistema detecta los 5 vectores de modificacion.
fail-closed en todos los casos. Ningun byte de plaintext
es expuesto cuando el contenedor fue manipulado.